

Introduction

Search a sorted array of 10 million numbers for one target value. Check every item one by one, and in the worst case you're looking at 10 million comparisons. Use binary search instead, and you're done in about 25.

That gap — 10,000,000 versus 25 — is the entire reason binary search shows up in nearly every technical interview and every algorithms course on day one. It's not just a clever trick; it's the cleanest possible demonstration of why the *shape* of an algorithm matters more than how fast the machine running it is. A faster computer makes linear search a little faster. A better algorithm makes it exponentially faster, and that gap only widens as the data grows.

Interviewers lean on binary search because it tests three things at once: whether you can reason about a precondition (the data must be sorted), whether you can translate "throw away half" into correct loop boundaries without off-by-one bugs, and whether you actually understand *why* it works rather than having memorized a template. This article walks through all three — starting from intuition, not code — so you leave with a mental model you can rebuild from scratch, in an interview or anywhere else.

Problem Overview

You're given a list of numbers sorted from smallest to largest, and a target value. Your job is to return the index of that target if it exists in the list, or signal clearly that it doesn't (typically by returning `-1`).

The one condition that makes this problem interesting is that **the list must already be sorted**. If it isn't, there's no reliable shortcut — you're back to checking items one at a time, because nothing tells you which direction the target might be in.

Example

Consider this sorted array of 10 integers:

index:	0	1	2	3	4	5	6	7	8	9
value:	2	5	8	13	21	27	33	41	50	67

Say we're searching for `33`. A linear scan checks index 0, then 1, then 2... all the way until it hits index 6. That's 7 comparisons.

Binary search instead starts by looking at the *middle* of the array. If the middle value is bigger than the target, the target — if it exists — has to be somewhere in the left half, because the array is sorted. If the middle value is smaller, the target has to be in the right half. Either way, half the array gets eliminated without inspecting a single element in it.

For `33` in the array above, binary search finds it in 2 comparisons instead of 7. On a 10-million-item array, that same principle brings a worst case of 10 million comparisons down to about 25 — because each comparison doesn't just eliminate one item, it eliminates half of whatever's left.

Intuition

Here's how an experienced engineer arrives at this solution without memorizing anything.

Start with the observation that scanning one item at a time wastes information. Every time you check `array[3]` and it's not the target, a linear scan learns exactly one fact: index 3 isn't it. Nothing more.

But if the array is sorted, checking the *middle* element tells you something much bigger. Suppose the middle value is smaller than your target. Because everything to the left of the middle is even smaller (sorted, remember), you now know with certainty that *none* of those elements can be the target either. You didn't just rule out one index — you ruled out half the array in a single comparison.

Repeat that logic on the remaining half. Check its middle. Rule out another half. And again. Each round the search space shrinks by half, which means the number of rounds needed is however many times you can cut a number in half before reaching 1 — that's exactly what a logarithm measures. Cutting 10,000,000 in half repeatedly hits 1 in about 24 steps. That's where the "25 steps" comes from.

The mental picture that makes this stick: a phone book. You don't start at "Aaronson" and read every name looking for "Mitchell." You open to the middle, see you're past or before where "Mitchell" would be, and flip to the half that could still contain it. Nobody taught you that consciously — it's exactly binary search.

The one thing this entire idea depends on is order. In a phone book, that's alphabetical order. In an array, that's sorted order. Take that away — a junk drawer instead of a phone book — and the middle element tells you nothing about which half to search, because "smaller" and "larger" no longer point anywhere useful.

Brute Force Solution

Idea: Walk through the array from the first element to the last, comparing each one to the target.

Algorithm: 1. Start at index 0. 2. Compare the current element to the target. 3. If it matches, return the current index. 4. Otherwise, move to the next index and repeat. 5. If you reach the end without a match, return -1.

Advantages: - Works on unsorted data — no precondition required. - Trivial to write and reason about.

Disadvantages: - Worst case checks every single element. - Doesn't take advantage of any structure in the data, even when structure (sorted order) is available.

Time complexity: $O(n)$ — worst case checks all n elements. **Space complexity:** $O(1)$ — no extra memory beyond a loop counter.

Optimal Solution

Binary search maintains two pointers, `low` and `high`, marking the current boundaries of the search space — initially the whole array.

Step by step:

1. Compute `mid = (low + high) // 2`.
2. Compare `array[mid]` to the target.
3. If they match, you're done — return `mid`.
4. If `array[mid]` is smaller than the target, the target (if present) must be to the right, so set `low = mid + 1`.
5. If `array[mid]` is larger than the target, the target must be to the left, so set `high = mid - 1`.
6. Repeat with the new `low / high` range.
7. Stop when `low > high` — at that point there's no range left to search, meaning the target isn't in the array.

Tracing this on the earlier array, searching for 33 :

Step low high mid array[mid] Comparison Action

1	0	9	4	21	$21 < 33$	<code>low = 5</code>
2	5	9	7	33	match	<code>return 7</code>

Two steps, not eight. Every round, `low` and `high` squeeze closer together, and the moment they cross (`low > high`), the search space is empty — that's the signal the value genuinely isn't in the array, not a bug.

Python Code

```
def binary_search(array: list[int], target: int) -> int:
    """Return the index of target in a sorted array, or -1 if absent."""
    low, high = 0, len(array) - 1

    while low <= high:
        mid = (low + high) // 2

        if array[mid] == target:
            return mid
        elif array[mid] < target:
            low = mid + 1
        else:
            high = mid - 1

    return -1
```

Code Walkthrough

- `low, high = 0, len(array) - 1` — the search space starts as the entire array.
- `while low <= high` — the loop continues as long as there's a valid range to check. The moment `low` slides past `high`, the range is empty and the loop exits.
- `mid = (low + high) // 2` — integer division picks the middle index of the current range.
- `if array[mid] == target: return mid` — found it, stop immediately.
- `elif array[mid] < target: low = mid + 1` — the target must be strictly to the right of `mid`, so the new range excludes `mid` entirely.
- `else: high = mid - 1` — the target must be strictly to the left of `mid`.
- `return -1` — reached only when the loop exits without finding a match, meaning the target isn't present.

Dry Run

Search for `33` in `[2, 5, 8, 13, 21, 27, 33, 41, 50, 67]`:

- `low=0, high=9` → `mid=4` → `array[4]=21` → `21 < 33` → `low=5`
- `low=5, high=9` → `mid=7` → `array[7]=33` → match → return `7`

Now search for `100`, which isn't in the array:

- `low=0, high=9` → `mid=4` → `21 < 100` → `low=5`

- `low=5, high=9` → `mid=7` → `41 < 100` → `low=8`
- `low=8, high=9` → `mid=8` → `50 < 100` → `low=9`
- `low=9, high=9` → `mid=9` → `67 < 100` → `low=10`
- `low=10, high=9` → loop condition `low <= high` fails → return `-1`

Complexity Analysis

Time: $O(\log n)$. Each iteration cuts the search range in half. Starting from n elements, the number of halvings needed to shrink the range to a single element is $\log_2(n)$. For $n = 10,000,000$, $\log_2(n) \approx 23-24$, matching the "about 25 steps" from the video.

Space: $O(1)$ for the iterative version above — only a fixed number of variables (`low`, `high`, `mid`) regardless of array size. (A recursive version would use $O(\log n)$ space for the call stack.)

These bounds are tight, not approximate: every iteration strictly shrinks the range, and the range can only shrink $\log_2(n)$ times before hitting a single element, so the algorithm can't do better or worse than this asymptotically.

Alternative Solutions

Recursive binary search expresses the same logic as a function calling itself on a smaller range instead of looping. Same $O(\log n)$ time, but $O(\log n)$ space due to the call stack — worth mentioning in an interview as a trade-off, but the iterative version is generally preferred in production code to avoid stack depth concerns.

Using **`bisect`** from Python's standard library (`bisect.bisect_left`) does the same job for real-world code, since it's implemented in C and well-tested. Writing binary search by hand is still worth knowing cold, though — it's less about the specific problem and more about the boundary-handling skill it tests.

Edge Cases

- **Empty array** — `low=0, high=-1`, loop never executes, returns `-1` immediately. Correct.
- **Single-element array** — `low=0, high=0`, one comparison, resolves either way.
- **Target is the first or last element** — works correctly since the range naturally narrows toward either end.
- **Duplicate values** — this version returns *some* valid index matching the target, not necessarily the first or last occurrence. If you need the first/last occurrence specifically, that requires a

modified binary search that keeps narrowing even after finding a match.

- **Target smaller than every element or larger than every element** — the range shrinks toward one edge and then crosses, correctly returning `-1`.

Common Mistakes

1. **Using `mid` instead of `mid + 1` or `mid - 1` when updating bounds.** Setting `low = mid` instead of `low = mid + 1` can cause an infinite loop, since the range never actually shrinks.
2. **Running binary search on unsorted data.** It won't error — it will just silently return wrong results, which is far more dangerous than a crash.
3. **Off-by-one errors in the loop condition**, using `low < high` instead of `low <= high`, which misses the case where the range has exactly one element left to check.
4. **Integer overflow when computing `mid`** in languages with fixed-size integers (`(low + high) // 2` can overflow for very large arrays). Not a practical issue in Python, but a classic interview follow-up — the fix is `mid = low + (high - low) // 2`.
5. **Forgetting to return `-1` (or an equivalent signal)** and instead letting the function fall through and return `None` implicitly, which can cause confusing bugs downstream if the caller expects an integer.

Interview Questions

- How would you modify this to find the *first* or *last* occurrence of a duplicate target?
- How would you binary search on a rotated sorted array?
- Can you write this recursively, and what's the space trade-off?
- What's the smallest and largest possible number of iterations for a given array size?
- How would binary search apply to a problem where the array itself isn't sorted, but some derived property of it is monotonic (binary search on the answer)?

Similar Problems

- **Search in Rotated Sorted Array (LeetCode 33)** — same binary search skeleton, adapted to handle a rotation point.
- **Find First and Last Position of Element in Sorted Array (LeetCode 34)** — a direct extension of the duplicate-handling edge case above.
- **Search Insert Position (LeetCode 35)** — binary search where the "not found" case still needs a meaningful answer (where the value would be inserted).

- **Koko Eating Bananas (LeetCode 875)** and **Capacity to Ship Packages Within D Days (LeetCode 1011)** — "binary search on the answer" problems, where you binary search over a range of possible answers rather than over an array.

Key Takeaways

- Binary search only works on sorted data — that precondition is not optional.
- Each comparison eliminates half the remaining search space, which is why the total steps scale with $\log(n)$ instead of n .
- The loop terminates correctly when `low > high`, which is your signal that the target doesn't exist.
- Almost every bug in a hand-written binary search comes from boundary handling — get `mid + 1` / `mid - 1` and `<=` right, and the rest follows from the intuition.

Watch the Video

For the full walkthrough — including tracing the algorithm by hand and running it against the code live — watch the video here: {LINK}

About the Series

This article is part of *Fun with Learning Technology*, a series focused on tracing algorithms and code until they actually make sense — not just copying a solution that works, but understanding why it works well enough to rebuild it from scratch under pressure.

Call To Action

If this cleared up binary search for you, the video walks through the trace and live run side by side — give it a watch and a like on YouTube. Subscribe there for more of these breakdowns, and subscribe to the Substack for the written companion pieces. Drop a comment with the trickiest boundary bug you've hit in your own binary search, and share this with anyone prepping for interviews.

Quick Review

See 'search a sorted array for a target' — which pattern?

Binary search: repeatedly halve the sorted range instead of scanning linearly.

Time complexity of binary search on n items?

$O(\log n)$ — each step discards half the remaining range.

Space complexity of iterative binary search?

$O(1)$ — just low/high/mid pointers, no extra structures.

Biggest bug source when computing mid index?

$\text{low} + (\text{high} - \text{low}) // 2$ vs $(\text{low} + \text{high}) // 2$; also forgetting array must be sorted first.

How do you know the target isn't in the array?

Loop ends when $\text{low} > \text{high}$ without a match — return -1/None, don't assume it's present.

Follow-up: why does binary search beat linear scan at scale?

10M items: ~24 comparisons vs up to 10M — logarithmic growth stays fast as n grows.